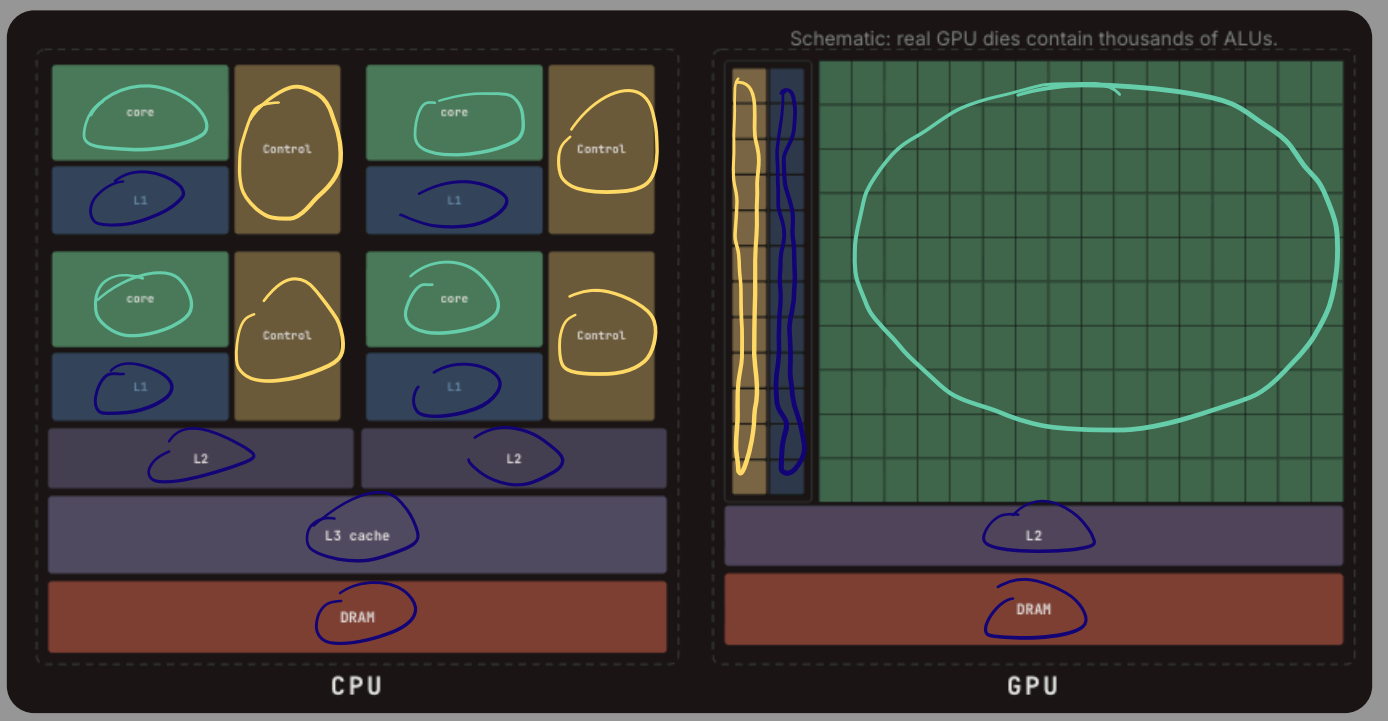
→ CPU vs. GPU



Arithmetic Logic Unit: ALUs do the arithmetic.

CPU has few large ALUs per core

GPU tiles the die w/ loads of smaller ones for parallel thruput

Per-thread control: fetch, decode, branches, scheduling

CPUs replicate heavy control per core for branchy code

GPUs use SIMT: one instruction for many threads control stays thin

Cache and Memory:

CPUs devote huge area to L3/L2/L1 so one thread hides DRAM latency

GPUs keep little on-chip cache and hide latency by switching warps instead

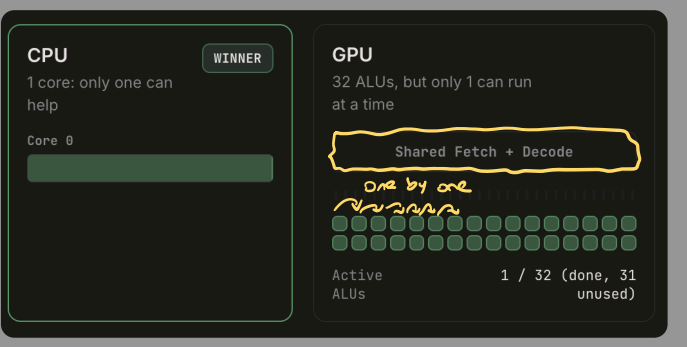
◈ CPUs spend area on cache and control for a few wide, sequential cores. GPUs spend it on ALU density and lean control for massive parallelism.

=> DIFFERENT FUNCTIONALITIES:

> Running Sum

$$y[i] = y[i-1] + x[i]$$

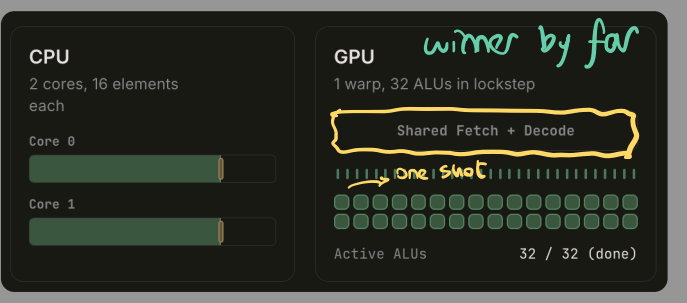
<> each element depends on the last one



> Multiply every element

$$y[i] = 2 * x[i]$$

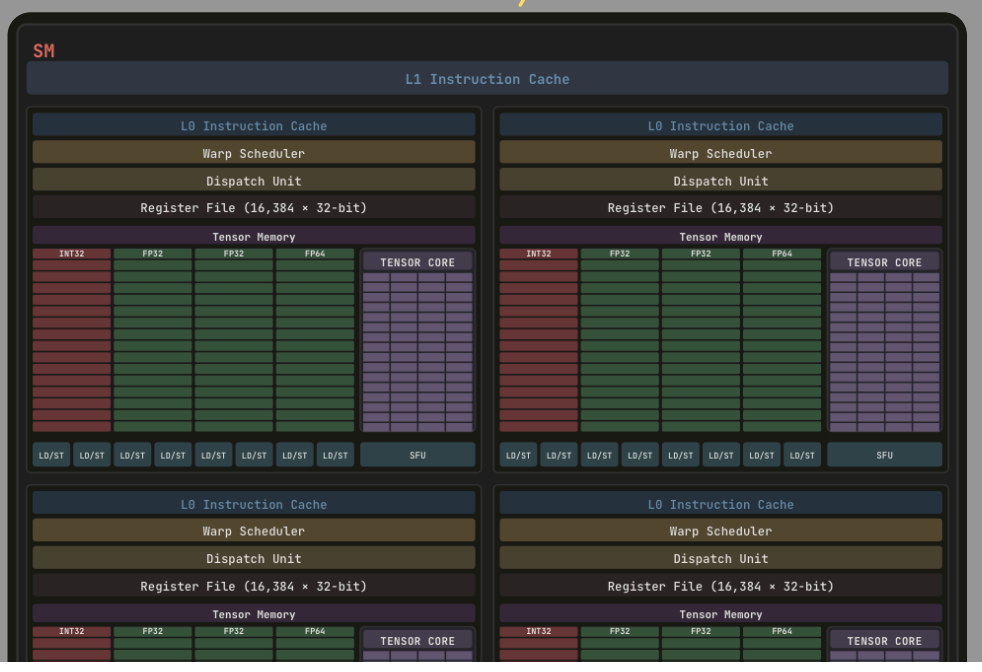
<> every element is independent



When every element is independent, the GPU fires all 32 ALUs at the same time. The CPU has to sequence through 16 elements per core. The GPU finishes in one flash.

-> The Workhorse.

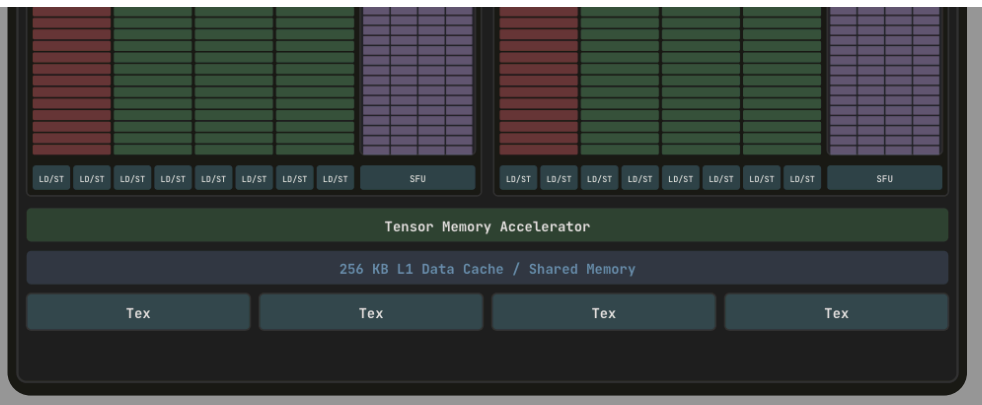
=> Streaming Multiprocessor



=> HHD style partitioning

NVIDIA's unit of execution Streaming Multiprocessor (SM):

- runs warps of threads
- keeps per-thread state in a large register file
- hides latency by switching warps every cycle when operands are ready not by huge speculative CPU machinery



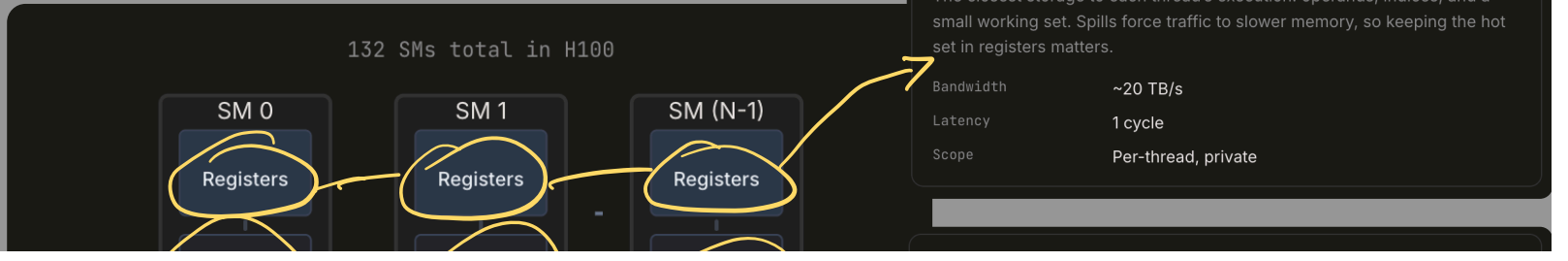
→ The full chip.

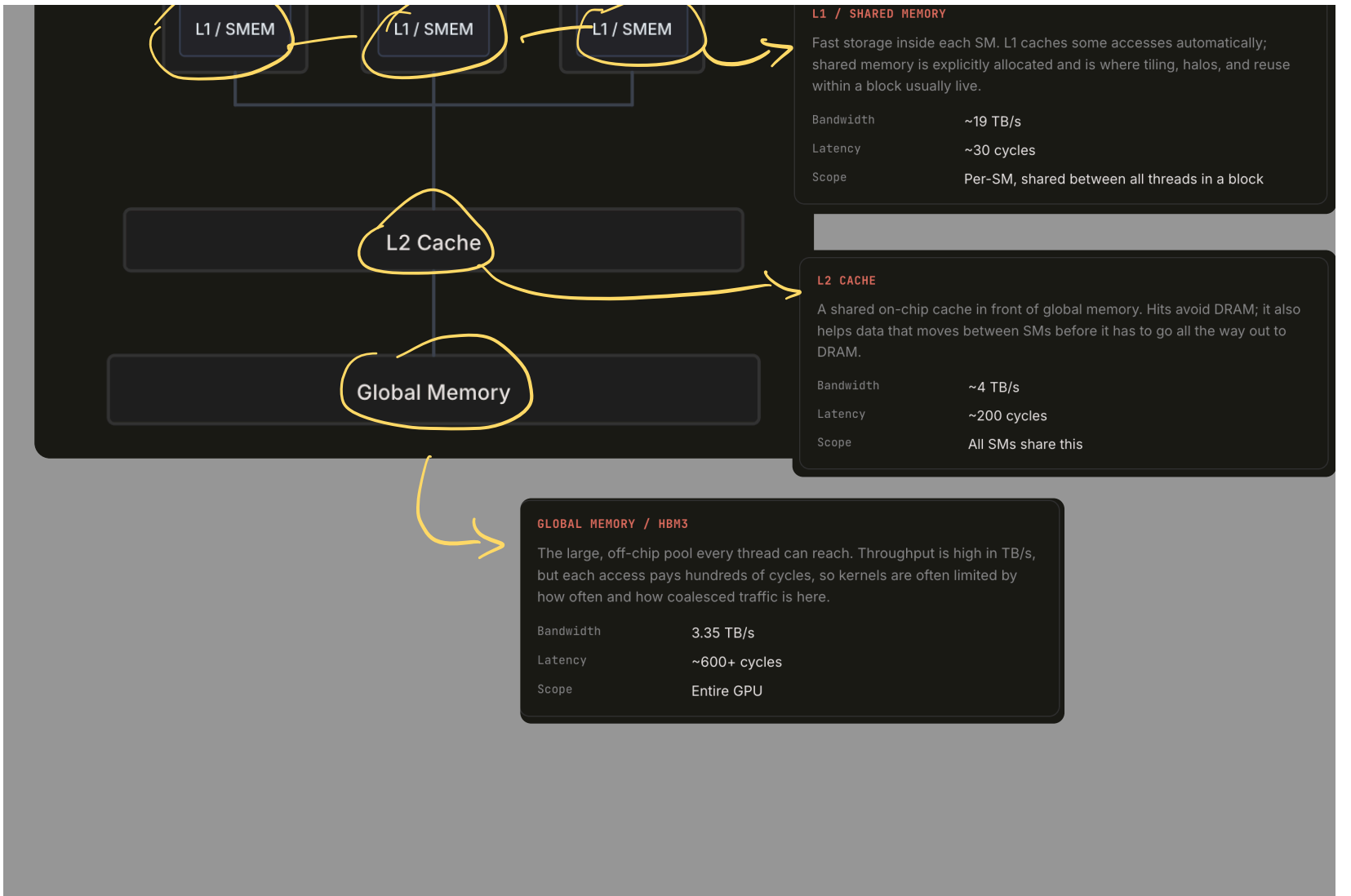
⇒ Device Architecture

→ each tile is one SM (132 SM total)

L2 and VRAM below are shared by the whole chip.

⇒ Memory Hierarchy




L1 / SMEM
L1 / SMEM
L1 / SMEM
L2 Cache
Global Memory
L1 / SHARED MEMORY
Fast storage inside each SM. L1 caches some accesses automatically; shared memory is explicitly allocated and is where tiling, halos, and reuse within a block usually live.
Bandwidth ~19 TB/s
Latency ~30 cycles
Scope Per-SM, shared between all threads in a block
L2 CACHE
A shared on-chip cache in front of global memory. Hits avoid DRAM; it also helps data that moves between SMs before it has to go all the way out to DRAM.
Bandwidth ~4 TB/s
Latency ~200 cycles
Scope All SMs share this
GLOBAL MEMORY / HBM3
The large, off-chip pool every thread can reach. Throughput is high in TB/s, but each access pays hundreds of cycles, so kernels are often limited by how often and how coalesced traffic is here.
Bandwidth 3.35 TB/s
Latency ~600+ cycles
Scope Entire GPU